AMDLoraTrainer: FLUX Klein LoRA training, steps 2 to 9

All the code for the image-model path: format detector, int8 convrot layers, FLUX Klein loader, text encoder, image dataset, cache, training loop and ComfyUI-format save. Apply the steps in order. Each step ends with a check you can run.

How to read this document

- **NEW FILE** blocks: create the file at that path with exactly that content. Two of them (`data/image_dataset.py`, `data/cache.py`) replace a one-line placeholder that already exists.
- **EDIT** blocks: the code under "find this" is in the file now; the code under "replace it with this" is what it becomes. Some edits say "add at the end of the file" instead.
- Paths are relative to the project root `E:\zi\Web Projects\AMDLoraTrainer`.
- Indentation matters in Python. Some PDF viewers drop leading spaces when you copy, so check the indentation after pasting. New files use 4 spaces. Edits to your existing files keep that file's own indentation (2 spaces in inject.py, loader.py, registry.py and save.py).

What was run before writing this, and what was not

Check	Result
Format detector on your 4 real files	Klein: int8_convrot, 172 int8 layers, 13 float. Text encoder: int8_convrot, 253 and 105. Both VAE files: plain.
Load int8 text encoder and encode a prompt	Worked. 12 s to load, 3.8 GB VRAM, output shape [80, 20480].
Load int8 transformer and generate a 512 px image	Worked. Correct image (a fox in snow). 22 s to load, 5.5 GB VRAM, 20 steps in 23 s, 6.3 GB peak.
Training run: 8 synthetic images, 42 updates, 512 px, rank 16	Worked. 1.7 s per update, 7.3 to 7.5 GB peak VRAM, loss fell, sample image moved toward the training images.
Save, reload, generate again	Reloaded LoRA gave a pixel-identical image to the in-training sample.
Saved key names	All 228 saved modules match ComfyUI's Flux3 LoRA key rule and the diffusers converter.
Unit tests	102 passed: your existing tests plus 28 new ones.
Not run	Loading the LoRA inside ComfyUI itself. 1024 px training. Training with gradient checkpointing off. The plain-weights path (a diffusers folder as base_model). A real photo dataset.

Three design points that differ from what we discussed

- **q, k and v stay fused.** Your file stores them as one layer (qkv). diffusers supports a fused layer (to_qkv), so the loader uses that instead of splitting rows. The LoRA is trained on the fused layer and **saved** split into q_proj, k_proj and v_proj, which is the key form ComfyUI and diffusers both read.
- **No VAE key map.** The VAE loads from the diffusers-format file you downloaded.

- **The VAE runs with plain attention.** With your AMD environment variables the VAE's single 512-wide attention head failed in flash attention ("profiler is not initialized"), so the VAE calls force the plain math path. The transformer and text encoder are not affected.

One value is my own choice, not from the model's documentation: `timestep_shift: 3.0` for training. The model samples with shift 6.93, and I found no published training recipe. 3.0 is a common LoRA setting; treat it as a knob to tune.

Order

Step	What	Files
2	Format detector	models/formats.py (new), config.py, models/loader.py, tests/test_formats.py (new)
3	Int8 layers	models/quant.py (new), lora/inject.py, tests/test_quant.py (new)
4 + 5	Klein transformer, text encoder, VAE, recipe	models/flux_klein.py (new), models/registry.py, models/loader.py, tests/test_flux_klein.py (new)
6	Image dataset and buckets	data/image_dataset.py (replace), config.py
7	Cache	data/cache.py (replace)
8	Training loop	train/diffusion.py (new), train/loop.py, configs/flux_klein_512.yaml (new)
9	Save for ComfyUI, sample script	io/save.py, scripts/sample_flux.py (new), tests/test_save_comfy.py (new), tests/test_image_dataset.py (new)

Steps 8 and 9 depend on each other (the loop imports the save function), so run the training check after step 9.

Step 2: format detector

Reads only the safetensors header and reports `plain`, `int8_convrot` or `fp8`. GGUF, nvfp4, mxfp8 and unlabeled int8 raise `UnsupportedFormat`.

NEW FILE: src/amdlora/models/formats.py

```
"""Weight-format detector: reads a safetensors header (never the weights) and says how the
weights are stored.

    plain         ordinary float weights (bf16 / fp16 / fp32)
    int8_convrot  ComfyUI int8 weights, one scale per row, Hadamard-rotated inputs
    fp8           8-bit float weights (detected, loader comes later)

GGUF, nvfp4, mxfp8 and int8 without a label raise UnsupportedFormat.
"""
from __future__ import annotations
import json
import struct
from collections import Counter
from dataclasses import dataclass, field
from pathlib import Path

FLOAT_DTYPES = {"F32", "F16", "BF16"}
FP8_DTYPES = {"F8_E4M3", "F8_E5M2"}
QUANT_SUFFIX = ".comfy_quant"


class UnsupportedFormat(ValueError):
    """The file is valid, but this trainer cannot train on its weight format."""


@dataclass
class FormatReport:
    kind: str                                   # plain | int8_convrot | fp8
    float_dtype: str | None = None              # most common dtype of the float 2-D weights
    quant_layers: dict[str, dict] = field(default_factory=dict)   # layer prefix -> its label
    float_layers: list[str] = field(default_factory=list)         # prefixes of float 2-D weights
    groupsize: int | None = None                # convrot group size
    metadata: dict = field(default_factory=dict)


def read_header(path) -> tuple[dict, dict, int]:
    """Returns (tensors, metadata, data_start). tensors maps name -> {dtype, shape, data_offsets}."""
    path = Path(path)
    size = path.stat().st_size
    with open(path, "rb") as f:
        first = f.read(8)
        if len(first) < 8:
            raise ValueError(f"{path.name}: too small to be a safetensors file")
        n = struct.unpack("<Q", first)[0]
        if n > size - 8:
            raise ValueError(f"{path.name}: not a safetensors file (header length {n} > file size)")
```

```
        tensors = json.loads(f.read(n))
    metadata = tensors.pop("__metadata__", None) or {}
    return tensors, metadata, 8 + n


def read_raw(path, entry: dict, data_start: int) -> bytes:
    """Raw bytes of one tensor. Only used for the tiny label tensors."""
    start, end = entry["data_offsets"]
    with open(path, "rb") as f:
        f.seek(data_start + start)
        return f.read(end - start)


def _is_gguf(path: Path) -> bool:
    if path.suffix.lower() == ".gguf":
        return True
    with open(path, "rb") as f:
        return f.read(4) == b"GGUF"


def detect_format(path) -> FormatReport:
    path = Path(path)
    if _is_gguf(path):
        raise UnsupportedFormat("GGUF can't be used as a base model; supply the safetensors version.")

    tensors, metadata, data_start = read_header(path)
    report = FormatReport(kind="plain", metadata=metadata)

    # ---- quantized layers: every one carries a small JSON label ----
    label_keys = [k for k in tensors if k.endswith(QUANT_SUFFIX)]
    if label_keys:
        with open(path, "rb") as f:                     # one open for all labels
            for key in label_keys:
                start, end = tensors[key]["data_offsets"]
                f.seek(data_start + start)
                report.quant_layers[key[:-len(QUANT_SUFFIX)]] = json.loads(f.read(end - start))

        groupsizes = set()
        for prefix, label in report.quant_layers.items():
            fmt = str(label.get("format", ""))
            if not (fmt.startswith("int8") and label.get("convrot")):
                raise UnsupportedFormat(f"{path.name}: layer '{prefix}' uses format '{fmt}' "
                                        f"(convrot={label.get('convrot')}), which is not supported")
            groupsizes.add(int(label.get("convrot_groupsize", 256)))
        if len(groupsizes) != 1:
            raise UnsupportedFormat(f"{path.name}: mixed convrot group sizes {sorted(groupsizes)}")
        report.kind = "int8_convrot"
        report.groupsize = groupsizes.pop()

        for prefix in report.quant_layers:
            weight = tensors.get(prefix + ".weight")
            scale = tensors.get(prefix + ".weight_scale")
            if weight is None or weight["dtype"] != "I8" or len(weight["shape"]) != 2:
```

```
                raise ValueError(f"{path.name}: '{prefix}.weight' must be a 2-D int8 tensor")
            rows, cols = weight["shape"]
            if scale is None or scale["dtype"] != "F32" or scale["shape"] != [rows, 1]:
                raise ValueError(f"{path.name}: '{prefix}.weight_scale' must be float32 with shape [{rows}, 1]")
            if cols % report.groupsize != 0:
                raise ValueError(f"{path.name}: '{prefix}' has {cols} inputs, "
                                 f"not divisible by group size {report.groupsize}")

    # ---- every other weight with 2 or more dimensions ----
    float_counts: Counter = Counter()
    for key, entry in tensors.items():
        if not key.endswith(".weight") or len(entry["shape"]) < 2:
            continue
        prefix = key[:-len(".weight")]
        if prefix in report.quant_layers:
            continue
        dtype = entry["dtype"]
        if dtype in FP8_DTYPES:
            if report.kind == "int8_convrot":
                raise UnsupportedFormat(f"{path.name}: mixes int8 and fp8 weights")
            report.kind = "fp8"
        elif dtype == "I8":
            raise UnsupportedFormat(f"{path.name}: '{key}' is int8 but has no comfy_quant label")
        elif dtype not in FLOAT_DTYPES:
            raise UnsupportedFormat(f"{path.name}: '{key}' has dtype {dtype}, which is not supported")
        if len(entry["shape"]) == 2:
            report.float_layers.append(prefix)
            float_counts[dtype] += 1

    if float_counts:
        report.float_dtype = float_counts.most_common(1)[0][0]
    return report
```

EDIT: src/amdlora/config.py

Change 1 of 1: find this

```
    base_load: str = "bf16"       # bf16 | fp8 | nf4
```

and replace it with this

```
    base_load: str = "auto"       # auto (detect from the file) | bf16 | fp16 | fp32 | nf4
```

EDIT: src/amdlora/models/loader.py

Change 1 of 2: find this

```
ALLOWED = set(DTYPES) | {"nf4","fp8"}
```

and replace it with this

```
ALLOWED = set(DTYPES) | {"nf4","fp8","auto"}
```

Change 2 of 2: find this

```
  _check_precision(base_load)
  from transformers import AutoModelForCausalLM, AutoTokenizer
```

and replace it with this

```
  _check_precision(base_load)
  if base_load == "auto":
    base_load = "bf16"
  from transformers import AutoModelForCausalLM, AutoTokenizer
```

NEW FILE: tests/test_formats.py

```
import json
import pytest
import torch
from safetensors.torch import save_file
from amdlora.models.formats import detect_format, read_header, UnsupportedFormat
from amdlora.models.quant import quantize_int8_convrot


def label(**fields) -> torch.Tensor:
    return torch.tensor(list(json.dumps(fields).encode("utf-8")), dtype=torch.uint8)


def int8_layer(prefix: str, out: int = 8, inp: int = 16, group: int = 16) -> dict:
    q, scale = quantize_int8_convrot(torch.randn(out, inp), group)
    return {f"{prefix}.weight": q, f"{prefix}.weight_scale": scale,
            f"{prefix}.comfy_quant": label(format="int8_tensorwise", convrot=True, convrot_groupsize=group)}


def test_plain_file(tmp_path):
    path = tmp_path / "plain.safetensors"
    save_file({"a.weight": torch.randn(4, 4, dtype=torch.bfloat16), "a.bias": torch.zeros(4)}, str(path))
    report = detect_format(path)
    assert report.kind == "plain" and report.float_dtype == "BF16" and report.float_layers == ["a"]


def test_int8_convrot_file(tmp_path):
    path = tmp_path / "int8.safetensors"
    save_file(int8_layer("block.qkv"), str(path))
    report = detect_format(path)
    assert report.kind == "int8_convrot" and report.groupsize == 16
    assert list(report.quant_layers) == ["block.qkv"] and report.float_layers == []


def test_mixed_file(tmp_path):
    path = tmp_path / "mixed.safetensors"
    save_file({**int8_layer("block.qkv"), "embed.weight": torch.randn(4, 4, dtype=torch.bfloat16)}, str(path))
    report = detect_format(path)
    assert report.kind == "int8_convrot" and report.float_layers == ["embed"] and len(report.quant_layers) == 1
def test_unknown_quant_format_raises(tmp_path):
```

```
    path = tmp_path / "nvfp4.safetensors"
    tensors = int8_layer("block.qkv")
    tensors["block.qkv.comfy_quant"] = label(format="nvfp4")
    save_file(tensors, str(path))
    with pytest.raises(UnsupportedFormat, match="nvfp4"):
        detect_format(path)


def test_gguf_raises(tmp_path):
    path = tmp_path / "model.safetensors"
    path.write_bytes(b"GGUF" + bytes(64))
    with pytest.raises(UnsupportedFormat, match="GGUF"):
        detect_format(path)


def test_missing_scale_raises(tmp_path):
    path = tmp_path / "broken.safetensors"
    tensors = int8_layer("block.qkv")
    del tensors["block.qkv.weight_scale"]
    save_file(tensors, str(path))
    with pytest.raises(ValueError, match="weight_scale"):
        detect_format(path)


def test_read_header_returns_metadata(tmp_path):
    path = tmp_path / "meta.safetensors"
    save_file({"a.weight": torch.zeros(2, 2)}, str(path), metadata={"rank": "4"})
    tensors, metadata, data_start = read_header(path)
    assert metadata == {"rank": "4"} and tensors["a.weight"]["shape"] == [2, 2] and data_start > 8
```

Check

```
pytest tests/test_formats.py tests/test_config.py tests/test_loader.py -q
python -c "from amdlora.models.formats import detect_format as d; r = d(r'E:\Comfy-Desktop\ComfyUI-Shared\models\diffusion_models\flux3-image-klein-base-int8-convrot.safeten
# expected: int8_convrot 172 13 256
```

test_formats.py imports from quant.py, so it only passes once step 3's quant.py exists.

Step 3: int8 convrot layers

`FrozenInt8Linear` keeps the weight as int8 on the GPU. It rotates the input, multiplies by the int8 weight and applies one scale per row. A small custom autograd function rebuilds the float weight during the backward pass, so no float copy of a weight is held between forward and backward. `FrozenInt8Embedding` does the same for the text encoder's token table, turning only the looked-up rows into floats.

NEW FILE: src/amdlora/models/quant.py

```
"""Frozen int8 "convrot" layers. The weights stay int8 on the GPU for the whole run.

Format (read from the comfy_kitchen spec, Apache-2.0):
    stored  = round(W @ H / scale)      int8, one float32 scale per output row
    H       = block-diagonal Hadamard rotation over groups of `group` input channels,
              symmetric and self-inverse (H @ H = identity)
so      W @ x  =  (stored * scale) @ (H @ x)
The layer rotates x, multiplies by the int8 weights, then applies the row scales.
"""
from __future__ import annotations
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

_H4 = [[1, 1, 1, -1], [1, 1, -1, 1], [1, -1, 1, 1], [-1, 1, 1, 1]]


def build_hadamard(size: int) -> torch.Tensor:
    """Normalized regular Hadamard matrix, float32, shape [size, size]. size is a power of 4."""
    power = round(math.log(size, 4))
    if size < 4 or 4 ** power != size:
        raise ValueError(f"Hadamard size must be a power of 4, got {size}")
    h4 = torch.tensor(_H4, dtype=torch.float32)
    h = h4
    for _ in range(power - 1):
        h = torch.kron(h, h4)
    return h / math.sqrt(size)


def rotate(x: torch.Tensor, h: torch.Tensor, group: int) -> torch.Tensor:
    """Rotates the last dimension of x in groups of `group` channels."""
    shape = x.shape
    if shape[-1] % group != 0:
        raise ValueError(f"last dimension {shape[-1]} is not divisible by group size {group}")
    x = x.reshape(*shape[:-1], shape[-1] // group, group)
    return torch.matmul(x, h.to(x.dtype)).reshape(shape)


class _Int8Matmul(torch.autograd.Function):
    """y = x @ q.T with q int8. The backward pass rebuilds the float weights from the int8
    buffer, so autograd never keeps a float copy of a weight alive between forward and backward."""
    @staticmethod
```

```
    def forward(ctx, x, qweight):
        ctx.qweight = qweight
        return F.linear(x, qweight.to(x.dtype))

    @staticmethod
    def backward(ctx, grad_out):
        return torch.matmul(grad_out, ctx.qweight.to(grad_out.dtype)), None


class FrozenInt8Linear(nn.Module):
    """Drop-in for a frozen nn.Linear whose weight is stored as int8 convrot."""

    def __init__(self, qweight: torch.Tensor, weight_scale: torch.Tensor, group: int = 256,
                 bias: torch.Tensor | None = None):
        super().__init__()
        if qweight.dtype != torch.int8 or qweight.dim() != 2:
            raise ValueError("qweight must be a 2-D int8 tensor")
        self.out_features, self.in_features = qweight.shape
        if weight_scale.numel() != self.out_features:
            raise ValueError("weight_scale needs one value per output row")
        if self.in_features % group != 0:
            raise ValueError(f"in_features {self.in_features} is not divisible by group size {group}")
        self.group = group
        self.register_buffer("qweight", qweight)
        # buffers named *_f32 must stay float32: never call .to(dtype) / .half() on these modules
        self.register_buffer("scale_f32", weight_scale.reshape(-1).to(torch.float32))
        self.register_buffer("hadamard_f32", build_hadamard(group).to(qweight.device), persistent=False)
        self.register_buffer("bias", bias)

    @property
    def weight(self) -> torch.Tensor:
        """The int8 buffer. LoRALinear only reads `.weight.device` from it."""
        return self.qweight

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        y = _Int8Matmul.apply(rotate(x, self.hadamard_f32, self.group), self.qweight)
        y = y * self.scale_f32.to(y.dtype)
        if self.bias is not None:
            y = y + self.bias.to(y.dtype)
        return y

    @torch.no_grad()
    def dequantize(self, dtype: torch.dtype = torch.float32) -> torch.Tensor:
        """The original (un-rotated) float weight. For tests and merging, not for training."""
        w = self.qweight.to(torch.float32) * self.scale_f32[:, None]
        return rotate(w, self.hadamard_f32, self.group).to(dtype)

    def extra_repr(self) -> str:
        return f"in_features={self.in_features}, out_features={self.out_features}, int8 convrot group={self.group}"


class FrozenInt8Embedding(nn.Module):
    """Drop-in for a frozen nn.Embedding stored as int8 convrot. Only the looked-up rows are
```

```
    turned back into floats."""

    def __init__(self, qweight: torch.Tensor, weight_scale: torch.Tensor, group: int = 256,
                 dtype: torch.dtype = torch.bfloat16):
        super().__init__()
        if qweight.dtype != torch.int8 or qweight.dim() != 2:
            raise ValueError("qweight must be a 2-D int8 tensor")
        self.num_embeddings, self.embedding_dim = qweight.shape
        self.group = group
        self.out_dtype = dtype
        self.register_buffer("qweight", qweight)
        self.register_buffer("scale_f32", weight_scale.reshape(-1).to(torch.float32))
        self.register_buffer("hadamard_f32", build_hadamard(group).to(qweight.device), persistent=False)

    @property
    def weight(self) -> torch.Tensor:
        return self.qweight

    def forward(self, ids: torch.Tensor) -> torch.Tensor:
        rows = self.qweight[ids].to(torch.float32) * self.scale_f32[ids][..., None]
        return rotate(rows, self.hadamard_f32, self.group).to(self.out_dtype)


def quantize_int8_convrot(weight: torch.Tensor, group: int = 256) -> tuple[torch.Tensor, torch.Tensor]:
    """Float weight -> (int8 weight, float32 row scales). Used by the tests to build tiny files."""
    rotated = rotate(weight.to(torch.float32), build_hadamard(group).to(weight.device), group)
    scale = rotated.abs().amax(dim=1, keepdim=True).clamp_min(1e-12) / 127.0
    q = torch.round(rotated / scale).clamp_(-127, 127).to(torch.int8)
    return q, scale
```

EDIT: src/amdlora/lora/inject.py

Change 1 of 3: find this

```
from .layer import LoRALinear
```

and replace it with this

```
from .layer import LoRALinear
from ..models.quant import FrozenInt8Linear

LINEAR_TYPES = (nn.Linear, FrozenInt8Linear)     # every layer type a LoRA can wrap
```

Change 2 of 3: find this

```
  lora_dtype: torch.dtype = torch.float32
) -> list[str]:
```

and replace it with this

```
  lora_dtype: torch.dtype = torch.float32,
  skip: list[str] | None = None
) -> list[str]:
```

Change 3 of 3: find this

```
    if isinstance(module,nn.Linear) and _matches(name,target_modules):
     found.append(name)
```

and replace it with this

```
    if isinstance(module,LINEAR_TYPES) and _matches(name,target_modules):
      if skip and any(part in name for part in skip):
        continue
      found.append(name)
```

`lora/layer.py` needs no change: the int8 layer exposes `in_features`, `out_features` and `weight.device`, which is all LoRALinear reads.

NEW FILE: tests/test_quant.py

```
import pytest
import torch
import torch.nn as nn
import torch.nn.functional as F
from amdlora.lora.inject import inject_lora
from amdlora.lora.layer import LoRALinear
from amdlora.models.quant import (FrozenInt8Embedding, FrozenInt8Linear, build_hadamard,
                                  quantize_int8_convrot, rotate)


def relative_error(a, b) -> float:
    return ((a - b).norm() / b.norm()).item()


def test_hadamard_is_symmetric_and_self_inverse():
    for size in (4, 16, 256):
        h = build_hadamard(size)
        assert torch.allclose(h, h.T)
        assert torch.allclose(h @ h, torch.eye(size), atol=1e-5)
    with pytest.raises(ValueError):
        build_hadamard(8)


def test_rotate_twice_gives_the_input_back():
    x = torch.randn(3, 5, 64)
    h = build_hadamard(16)
    assert torch.allclose(rotate(rotate(x, h, 16), h, 16), x, atol=1e-5)


def test_int8_linear_matches_the_float_layer():
    torch.manual_seed(0)
    weight = torch.randn(24, 64) * 0.05
    layer = FrozenInt8Linear(*quantize_int8_convrot(weight, 16), group=16)
    x = torch.randn(2, 7, 64)
    assert relative_error(layer(x), F.linear(x, weight)) < 0.02
    assert relative_error(layer.dequantize(), weight) < 0.02
```

```
    assert layer.weight.dtype == torch.int8 and (layer.in_features, layer.out_features) == (64, 24)


def test_int8_linear_gradient_matches_a_dense_layer():
    torch.manual_seed(0)
    layer = FrozenInt8Linear(*quantize_int8_convrot(torch.randn(24, 64), 16), group=16)
    x = torch.randn(2, 7, 64, requires_grad=True)
    grad, = torch.autograd.grad(layer(x).pow(2).sum(), x)
    dense, = torch.autograd.grad(F.linear(x, layer.dequantize()).pow(2).sum(), x)
    assert relative_error(grad, dense) < 1e-4


def test_int8_embedding_matches_the_float_table():
    torch.manual_seed(0)
    table = torch.randn(50, 32) * 0.02
    layer = FrozenInt8Embedding(*quantize_int8_convrot(table, 16), group=16, dtype=torch.float32)
    ids = torch.tensor([[1, 5], [49, 0]])
    assert layer(ids).shape == (2, 2, 32)
    assert relative_error(layer(ids), table[ids]) < 0.02


def test_lora_wraps_an_int8_layer_and_trains():
    torch.manual_seed(0)

    class Block(nn.Module):
        def __init__(self):
            super().__init__()
            self.to_qkv = FrozenInt8Linear(*quantize_int8_convrot(torch.randn(48, 32), 16), group=16)

        def forward(self, x):
            return self.to_qkv(x)

    model = nn.ModuleDict({"keep": Block(), "skip_me": Block()})
    names = inject_lora(model, ["to_qkv"], rank=4, skip=["skip_me"])
    assert names == ["keep.to_qkv"] and isinstance(model["keep"].to_qkv, LoRALinear)

    x = torch.randn(3, 32)
    before = model["keep"](x)
    assert torch.allclose(before, model["keep"].to_qkv.base(x))          # B starts at zero
    before.pow(2).sum().backward()
    lora = model["keep"].to_qkv
    assert lora.B.grad is not None and lora.B.grad.abs().sum() > 0
```

Check

```
pytest tests/test_quant.py tests/test_formats.py tests/test_inject.py tests/test_lora_layer.py -q
```

Steps 4 and 5: FLUX Klein transformer, text encoder, VAE and recipe

One module holds everything specific to this model family. A future family (Qwen-Image, Krea-2) is another module with the same function names plus one line in the registry.

- **Names:** ComfyUI layer names to diffusers names and back.
- **Loaders:** the model structure is built empty (no memory), int8 layers are put in place, float layers are assigned from the file. Every tensor in the file must land somewhere or the loader raises.
- **Text recipe:** chat template, padding to a multiple of 80 tokens, hidden states of layers 4, 8, ..., 32 joined to 20480 values per token.
- **Latent recipe:** VAE encode, 2x2 packing to 128 channels, batch-norm scaling, position ids.
- **Flow matching:** noisy = (1 - t) * image + t * noise, and the model predicts noise - image.

NEW FILE: src/amdlora/models/flux_klein.py

```
"""FLUX Klein family: everything that is specific to this model.

    names      ComfyUI layer names <-> diffusers layer names
    loaders    transformer (int8 convrot file or diffusers folder), text encoder, VAE
    recipe     prompt encoding, latent packing, position ids, flow-matching noise, sampler

The model recipe (8 hidden layers, 80-token padding, shift 6.93, position ids, VAE batch-norm
scaling) follows the public diffusers FLUX.3 pipeline and BFL's config.json.
"""
from __future__ import annotations
import json
import math
from pathlib import Path

import torch
from accelerate import init_empty_weights
from safetensors.torch import load_file
from torch.nn.attention import SDPBackend, sdpa_kernel

from .formats import detect_format, UnsupportedFormat
from .quant import FrozenInt8Embedding, FrozenInt8Linear

# ---- recipe constants (BFL config.json) ----
TEXT_LAYERS = (4, 8, 12, 16, 20, 24, 28, 32)   # hidden states that are joined: 8 x 2560 = 20480
PAD_MULTIPLE = 80                              # prompts are right-padded to a multiple of 80 tokens
MAX_TOKENS = 8192
IMAGE_ROPE_SCALE = 0.5
SAMPLE_SHIFT = 6.93
PIXELS_PER_TOKEN = 16                          # VAE shrinks 8x, then 2x2 latents become one token

LORA_TARGETS = ["attn.to_qkv", "attn.to_out.0", "ff.linear_in", "ff.linear_out"]
LORA_SKIP = ["mode_blocks.image_cond."]        # reference-image blocks never run in text-to-image
# ================================================================================
# names
```

```
# ============================================================================
_EXACT = {
    "txt_in": "context_embedder",
    "time_in.in_layer": "timestep_embedder.linear_1",
    "time_in.out_layer": "timestep_embedder.linear_2",
    "final_layer.image.linear": "proj_out",
    "final_layer.image.adaLN_modulation.1": "norm_out.linear",
}
_PREFIXES = {
    "emb_in.": "x_embedders.",
    "content_mode_blocks.": "mode_blocks.",
    "single_blocks.": "single_transformer_blocks.",
}
_SUFFIXES = {
    ".qkv": ".attn.to_qkv",
    ".attn_out": ".attn.to_out.0",
    ".mlp_in": ".ff.linear_in",
    ".mlp_out": ".ff.linear_out",
    ".norm.query_norm": ".attn.norm_q",
    ".norm.key_norm": ".attn.norm_k",
    ".lin": ".linear",
}


def to_diffusers_name(module: str) -> str:
    """ComfyUI module name -> diffusers module name ('single_blocks.3.qkv' -> '...attn.to_qkv')."""
    if module in _EXACT:
        return _EXACT[module]
    for old, new in _PREFIXES.items():
        if module.startswith(old):
            module = new + module[len(old):]
            break
    for old, new in _SUFFIXES.items():
        if module.endswith(old):
            module = module[:-len(old)] + new
            break
    return module


def to_comfy_name(module: str) -> str:
    """diffusers module name -> ComfyUI module name. Only block layers are needed (LoRA targets)."""
    for old, new in _PREFIXES.items():
        if module.startswith(new):
            module = old + module[len(new):]
            break
    for old, new in _SUFFIXES.items():
        if module.endswith(new):
            module = module[:-len(new)] + old
            break
    return module


def _set_module(model, name: str, module) -> None:
```

```
    parent_name, _, child = name.rpartition(".")
    parent = model.get_submodule(parent_name) if parent_name else model
    setattr(parent, child, module)


def _check_loaded(model, missing, unexpected, what: str) -> None:
    missing = [k for k in missing if not k.endswith((".qweight", ".scale_f32"))]   # int8 buffers, set already
    on_meta = [n for n, p in model.named_parameters() if p.is_meta]
    if missing or unexpected or on_meta:
        raise ValueError(f"{what}: the file does not match the model. missing={list(missing)[:4]} "
                         f"unexpected={list(unexpected)[:4]} unfilled={on_meta[:4]}")


# ============================================================================================
# loaders
# ============================================================================================
def load_transformer(path, config_dir, device="cuda", dtype=torch.bfloat16):
    """Returns a frozen Flux3Transformer2DModel.
    path: a ComfyUI int8 convrot .safetensors file, or a diffusers folder with plain weights."""
    from diffusers import Flux3Transformer2DModel

    path = Path(path)
    if path.is_dir():
        model = Flux3Transformer2DModel.from_pretrained(str(path), torch_dtype=dtype).to(device)
        return model.requires_grad_(False).eval()

    report = detect_format(path)
    if report.kind != "int8_convrot":
        raise UnsupportedFormat(f"{path.name}: format '{report.kind}'. For FLUX Klein use the int8 convrot "
                                f"file or a diffusers folder.")

    config = json.loads((Path(config_dir) / "transformer" / "config.json").read_text(encoding="utf-8"))
    with init_empty_weights():                      # build the structure without allocating the weights
        model = Flux3Transformer2DModel.from_config(config)

    tensors = load_file(str(path), device=str(device))

    # 1. int8 layers
    for prefix in report.quant_layers:
        layer = FrozenInt8Linear(tensors.pop(prefix + ".weight"), tensors.pop(prefix + ".weight_scale"),
                                 report.groupsize)
        tensors.pop(prefix + ".comfy_quant")
        target = to_diffusers_name(prefix)
        if target.endswith(".attn.to_qkv"):          # the file stores q, k and v as one fused layer
            attn = model.get_submodule(target[:-len(".to_qkv")])
            del attn.to_q, attn.to_k, attn.to_v
            attn.to_qkv = layer
            attn.fused_projections = True
        else:
            _set_module(model, target, layer)

    # 2. float layers
    state = {}
```

```
    for key, value in tensors.items():
        if key.endswith(".comfy_attention.config"):
            continue
        module, param = key.rsplit(".", 1)
        new_key = to_diffusers_name(module) + "." + ("weight" if param == "scale" else param)
        if new_key == "norm_out.linear.weight":      # file order is [shift, scale]; diffusers wants [scale, shift]
            shift, scale = value.chunk(2, dim=0)
            value = torch.cat([scale, shift], dim=0)
        is_qk_norm = ".attn.norm_q." in new_key or ".attn.norm_k." in new_key
        state[new_key] = value.to(torch.float32 if is_qk_norm else dtype)   # q/k norms run in float32
    missing, unexpected = model.load_state_dict(state, strict=False, assign=True)
    _check_loaded(model, missing, unexpected, path.name)
    return model.to(device).requires_grad_(False).eval()


def load_text_encoder(path, config_dir, device="cuda", dtype=torch.bfloat16):
    """Returns (text_encoder, tokenizer). path: the int8 convrot Qwen3-VL .safetensors file.
    Only the language model is built; the vision tower is not needed for text."""
    from transformers import AutoTokenizer, Qwen3VLConfig, Qwen3VLModel

    path = Path(path)
    report = detect_format(path)
    if report.kind != "int8_convrot":
        raise UnsupportedFormat(f"{path.name}: format '{report.kind}'. Use the int8 convrot text encoder file.")

    text_dir = Path(config_dir) / "text_encoder"
    config = Qwen3VLConfig.from_pretrained(str(text_dir))
    with init_empty_weights():
        model = Qwen3VLModel(config)
    model.visual = None                              # text only

    tensors = load_file(str(path), device=str(device))
    for prefix in report.quant_layers:
        qweight, scale = tensors.pop(prefix + ".weight"), tensors.pop(prefix + ".weight_scale")
        tensors.pop(prefix + ".comfy_quant")
        target = prefix.removeprefix("model.")
        if target.endswith("embed_tokens"):
            layer = FrozenInt8Embedding(qweight, scale, report.groupsize, dtype)
        else:
            layer = FrozenInt8Linear(qweight, scale, report.groupsize)
        _set_module(model, target, layer)

    state = {k.removeprefix("model."): v.to(dtype) for k, v in tensors.items() if not k.startswith("model.visual.")}
    del tensors
    missing, unexpected = model.load_state_dict(state, strict=False, assign=True)
    _check_loaded(model, missing, unexpected, path.name)

    tokenizer = AutoTokenizer.from_pretrained(str(text_dir))
    if not tokenizer.chat_template:                  # the template ships in its own file
        tokenizer.chat_template = json.loads((text_dir / "chat_template.json").read_text(encoding="utf-8"))["chat_template"]
    return model.to(device).requires_grad_(False).eval(), tokenizer
def load_vae(vae_dir, device="cuda"):
    """The VAE in diffusers format (config.json + diffusion_pytorch_model.safetensors), float32."""
```

```
    from diffusers import AutoencoderKLFlux2

    vae = AutoencoderKLFlux2.from_pretrained(str(vae_dir), torch_dtype=torch.float32).to(device)
    return vae.requires_grad_(False).eval()


# ================================================================================
# recipe: text
# ================================================================================
@torch.no_grad()
def encode_prompt(text_encoder, tokenizer, prompt: str, dtype=torch.bfloat16) -> torch.Tensor:
    """One prompt -> embeddings of shape [tokens, 20480]. tokens is a multiple of 80."""
    device = next(text_encoder.parameters()).device
    text = tokenizer.apply_chat_template([{"role": "user", "content": prompt}],
                                         tokenize=False, add_generation_prompt=True)
    real = tokenizer(text, return_tensors="pt", truncation=True, max_length=MAX_TOKENS)["input_ids"].shape[1]
    padded = min(math.ceil(real / PAD_MULTIPLE) * PAD_MULTIPLE, MAX_TOKENS)
    inputs = tokenizer(text, return_tensors="pt", padding="max_length", padding_side="right",
                       truncation=True, max_length=padded)
    input_ids = inputs["input_ids"].to(device)
    attention_mask = inputs["attention_mask"].to(device)
    # padding tokens get position 1, as in the environment the model was trained in
    position_ids = torch.arange(padded, device=device)[None].masked_fill(attention_mask == 0, 1)

    out = text_encoder(input_ids=input_ids, attention_mask=attention_mask, position_ids=position_ids,
                       output_hidden_states=True, use_cache=False)
    stacked = torch.stack([out.hidden_states[i] for i in TEXT_LAYERS], dim=1)      # [1, 8, tokens, 2560]
    batch, layers, tokens, hidden = stacked.shape
    return stacked.permute(0, 2, 1, 3).reshape(tokens, layers * hidden).to(dtype)


# ================================================================================
# recipe: latents
# ================================================================================
def _bn_stats(vae, device, dtype):
    mean = vae.bn.running_mean.view(1, -1, 1, 1).to(device, dtype)
    std = torch.sqrt(vae.bn.running_var.view(1, -1, 1, 1) + vae.config.batch_norm_eps).to(device, dtype)
    return mean, std


@torch.no_grad()
def encode_image(vae, pixels: torch.Tensor) -> torch.Tensor:
    """pixels [B, 3, H, W] in [-1, 1] -> normalized packed latents [B, 128, H/16, W/16], float32."""
    with sdpa_kernel([SDPBackend.MATH]):     # the VAE's single 512-wide attention head fails in AMD flash attention
        latents = vae.encode(pixels.to(vae.device, vae.dtype)).latent_dist.mode()
    b, c, h, w = latents.shape                                          # [B, 32, H/8, W/8]
    latents = latents.view(b, c, h // 2, 2, w // 2, 2).permute(0, 1, 3, 5, 2, 4).reshape(b, c * 4, h // 2, w // 2)
    mean, std = _bn_stats(vae, latents.device, latents.dtype)
    return (latents - mean) / std


@torch.no_grad()
def decode_latents(vae, latents: torch.Tensor) -> torch.Tensor:
```

```
    """Normalized packed latents [B, 128, h, w] -> pixels [B, 3, 16h, 16w] in [-1, 1]."""
    latents = latents.to(vae.device, vae.dtype)
    mean, std = _bn_stats(vae, latents.device, latents.dtype)
    latents = latents * std + mean
    b, c, h, w = latents.shape
    latents = latents.reshape(b, c // 4, 2, 2, h, w).permute(0, 1, 4, 2, 5, 3).reshape(b, c // 4, h * 2, w * 2)
    with sdpa_kernel([SDPBackend.MATH]):
        pixels = vae.decode(latents, return_dict=False)[0]
    return pixels.clamp(-1, 1)


def pack(latents: torch.Tensor) -> torch.Tensor:
    """[B, C, h, w] -> tokens [B, h*w, C]"""
    return latents.flatten(2).transpose(1, 2)


def unpack(tokens: torch.Tensor, h: int, w: int) -> torch.Tensor:
    """tokens [B, h*w, C] -> [B, C, h, w]"""
    return tokens.transpose(1, 2).reshape(tokens.shape[0], tokens.shape[2], h, w)


def image_ids(h: int, w: int, device) -> torch.Tensor:
    """Position ids (t, h, w, l) for an h x w token grid, shape [h*w, 4]."""
    ids = torch.cartesian_prod(torch.arange(1), torch.arange(h), torch.arange(w), torch.arange(1)).float()
    ids[:, 1:3] *= IMAGE_ROPE_SCALE
    return ids.to(device)


def text_ids(tokens: int, device) -> torch.Tensor:
    """Position ids for the text tokens, shape [tokens, 4]: text advances along the last axis."""
    return torch.cartesian_prod(torch.arange(1), torch.arange(1), torch.arange(1), torch.arange(tokens)).float().to(device)


# ============================================================================
# recipe: flow matching
# ============================================================================
def shift_time(t: torch.Tensor, shift: float) -> torch.Tensor:
    return shift * t / (1 + (shift - 1) * t)


def sample_timesteps(batch: int, shift: float, device, generator=None) -> torch.Tensor:
    """Training noise levels in (0, 1): logit-normal, then shifted toward more noise."""
    t = torch.sigmoid(torch.randn(batch, generator=generator))
    return shift_time(t, shift).to(device)


def add_noise(latents: torch.Tensor, noise: torch.Tensor, t: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
    """Returns (noisy latents, target). t = 0 is the clean image, t = 1 is pure noise.
    The model predicts the velocity, which is noise - image."""
    t = t.to(latents.dtype).view(-1, *([1] * (latents.dim() - 1)))
    return (1 - t) * latents + t * noise, noise - latents
def predict(transformer, noisy_tokens, prompt_embeds, t, h: int, w: int) -> torch.Tensor:
    """One transformer call. noisy_tokens [B, h*w, 128], prompt_embeds [B, tokens, 20480], t [B]."""
```

```
    dtype = transformer.dtype
    return transformer(
        hidden_states=noisy_tokens.to(dtype),
        encoder_hidden_states=prompt_embeds.to(dtype),
        timestep=t.to(dtype),
        img_ids=image_ids(h, w, noisy_tokens.device),
        txt_ids=text_ids(prompt_embeds.shape[1], noisy_tokens.device),
        return_dict=False,
    )[0]


@torch.no_grad()
def generate(transformer, prompt_embeds, negative_embeds=None, height: int = 512, width: int = 512,
             steps: int = 28, guidance: float = 3.5, seed: int = 0, shift: float = SAMPLE_SHIFT) -> torch.Tensor:
    """Euler sampler. prompt_embeds [tokens, 20480]. Returns normalized packed latents [1, 128, h, w]."""
    device = next(transformer.parameters()).device
    h, w = height // PIXELS_PER_TOKEN, width // PIXELS_PER_TOKEN
    noise = torch.randn(1, 128, h, w, generator=torch.Generator().manual_seed(seed), dtype=torch.float32)
    x = pack(noise).to(device)
    sigmas = shift_time(torch.linspace(1.0, 1.0 / steps, steps), shift).tolist() + [0.0]
    cond = prompt_embeds[None].to(device)
    uncond = negative_embeds[None].to(device) if negative_embeds is not None and guidance > 1.0 else None
    for now, nxt in zip(sigmas[:-1], sigmas[1:]):
        t = torch.full((1,), now, device=device)
        velocity = predict(transformer, x, cond, t, h, w).float()
        if uncond is not None:
            velocity_uncond = predict(transformer, x, uncond, t, h, w).float()
            velocity = velocity_uncond + guidance * (velocity - velocity_uncond)
        x = x + (nxt - now) * velocity
    return unpack(x, h, w)


def to_pil(pixels: torch.Tensor):
    """pixels [3, H, W] in [-1, 1] -> PIL image"""
    from PIL import Image
    array = ((pixels.float().clamp(-1, 1) + 1) * 127.5).round().byte().permute(1, 2, 0).cpu().numpy()
    return Image.fromarray(array)
```

EDIT: src/amdlora/models/registry.py

Change 1 of 1: find this

```
from __future__ import annotations
```

and replace it with this

```
from __future__ import annotations
import importlib
```

EDIT: src/amdlora/models/registry.py (add at the end of the file)

```
# Image-model families. Each one is a module with the same set of functions (see flux_klein.py):
# load_transformer, load_text_encoder, load_vae, encode_prompt, encode_image, decode_latents,
```

```
# pack, sample_timesteps, add_noise, predict, generate, to_comfy_name, LORA_TARGETS, LORA_SKIP.
DIFFUSION_FAMILIES: dict[str, str] = {
    "flux_klein": "amdlora.models.flux_klein",
}

def get_family(name: str):
  if name not in DIFFUSION_FAMILIES:
    raise ValueError(f"Unknown diffusion family '{name}'. Known: {list(DIFFUSION_FAMILIES)}")
  return importlib.import_module(DIFFUSION_FAMILIES[name])
```

EDIT: src/amdlora/models/loader.py

Change 1 of 2: find this

```
def load_diffusion(path:str , base_load:str = "bf16" , device: str = "cuda"):
  _check_precision(base_load)
  raise NotImplementedError("Diffusion model waits until it is fully confirmed from the hugging face")
```

and replace it with this

```
def load_diffusion(cfg, device: str = "cuda"):
  """Returns the frozen transformer of an image model. The weight format is detected from the file."""
  from .registry import get_family
  _check_precision(cfg.precision.base_load)
  family = get_family(cfg.diffusion.family)
  return family.load_transformer(cfg.base_model, cfg.diffusion.config_dir, device, DTYPES[cfg.precision.compute_dtype])
```

Change 2 of 2: find this

```
    return load_diffusion(cfg.base_load , cfg.precision.base_load)
```

and replace it with this

```
    return load_diffusion(cfg), None
```

The second change also fixes the bug on line 79 (it passed `cfg.base_load` as the path). `load_diffusion` reads `cfg.diffusion`, which is added in step 6.

NEW FILE: tests/test_flux_klein.py

```
import torch
from amdlora.models import flux_klein as fk
from amdlora.models.registry import get_family


def test_names_round_trip():
    pairs = {
        "single_blocks.3.qkv": "single_transformer_blocks.3.attn.to_qkv",
        "content_mode_blocks.image.0.attn_out": "mode_blocks.image.0.attn.to_out.0",
        "txt_mode_blocks.4.mlp_in": "txt_mode_blocks.4.ff.linear_in",
        "single_blocks.27.mlp_out": "single_transformer_blocks.27.ff.linear_out",
    }
    for comfy, diffusers in pairs.items():
```

```
        assert fk.to_diffusers_name(comfy) == diffusers
        assert fk.to_comfy_name(diffusers) == comfy


def test_float_layer_names():
    assert fk.to_diffusers_name("txt_in") == "context_embedder"
    assert fk.to_diffusers_name("emb_in.image") == "x_embedders.image"
    assert fk.to_diffusers_name("time_in.out_layer") == "timestep_embedder.linear_2"
    assert fk.to_diffusers_name("early_stream_modulations.txt.lin") == "early_stream_modulations.txt.linear"
    assert fk.to_diffusers_name("single_blocks.0.norm.query_norm") == "single_transformer_blocks.0.attn.norm_q"
    assert fk.to_diffusers_name("final_layer.image.adaLN_modulation.1") == "norm_out.linear"


def test_pack_and_unpack():
    latents = torch.randn(2, 128, 4, 6)
    tokens = fk.pack(latents)
    assert tokens.shape == (2, 24, 128)
    assert torch.equal(fk.unpack(tokens, 4, 6), latents)


def test_position_ids():
    ids = fk.image_ids(2, 3, "cpu")
    assert ids.shape == (6, 4) and ids[:, 0].eq(0).all() and ids[:, 3].eq(0).all()
    assert ids[-1].tolist() == [0.0, 0.5, 1.0, 0.0]                    # h and w are scaled by 0.5
    text = fk.text_ids(5, "cpu")
    assert text.shape == (5, 4) and text[:, 3].tolist() == [0.0, 1.0, 2.0, 3.0, 4.0]


def test_noise_recipe():
    assert fk.shift_time(torch.tensor(0.5), 1.0).item() == 0.5
    assert fk.shift_time(torch.tensor(0.5), 3.0).item() == 0.75
    t = fk.sample_timesteps(1000, 3.0, "cpu")
    assert t.min() > 0 and t.max() < 1 and t.mean() > 0.5

    latents, noise = torch.randn(2, 8, 4, 4), torch.randn(2, 8, 4, 4)
    noisy, target = fk.add_noise(latents, noise, torch.tensor([0.0, 1.0]))
    assert torch.equal(noisy[0], latents[0]) and torch.equal(noisy[1], noise[1])
    assert torch.equal(target, noise - latents)


def test_registry_returns_the_family():
    family = get_family("flux_klein")
    for name in ("load_transformer", "load_text_encoder", "load_vae", "encode_prompt", "encode_image",
                 "decode_latents", "pack", "sample_timesteps", "add_noise", "predict", "generate",
                 "to_comfy_name", "LORA_TARGETS", "LORA_SKIP"):
        assert hasattr(family, name), name
```

Check

```
pytest tests/test_flux_klein.py -q
```

The real-weights check (generate an image) uses `scripts/sample_flux.py` from step 9.

Step 6: image dataset and buckets

Each image needs a caption file with the same name (photo.png and photo.txt). Images are sorted into buckets of about resolution x resolution pixels, so wide and tall images keep their shape. The batch sampler only puts items with the same image shape and the same caption length into one batch.

EDIT: src/amdlora/config.py

Change 1 of 2: find this

```
@dataclass
class Config:
```

and replace it with this

```
@dataclass
class DiffusionCfg:
    """Image-model settings. Only read when model_family is "diffusion"."""
    family: str = "flux_klein"        # which entry of models/registry.py to use
    config_dir: str = ""              # folder with transformer/config.json and text_encoder/ (tokenizer)
    vae: str = ""                     # VAE folder in diffusers format
    text_encoder: str = ""            # text encoder .safetensors file
    resolution: int = 512             # images are bucketed to about resolution x resolution pixels
    cache_dir: str = ""               # "" = <output_dir>/cache
    timestep_shift: float = 3.0       # > 1 trains more on noisy steps (composition), 1.0 = no shift
    caption_dropout: float = 0.0      # share of batches trained with an empty caption
    sample_steps: int = 20
    sample_guidance: float = 3.5
    sample_seed: int = 0


@dataclass
class Config:
```

Change 2 of 2: find this

```
    monitor: MonitorCfg = field(default_factory=MonitorCfg)
```

and replace it with this

```
    monitor: MonitorCfg = field(default_factory=MonitorCfg)
    diffusion: DiffusionCfg = field(default_factory=DiffusionCfg)
```

The first change inserts the new `DiffusionCfg` class directly above `class Config`. The second adds one field at the end of `Config`.

NEW FILE: src/amdlora/data/image_dataset.py

```
"""Step 6: image + same-name .txt caption pairs, sorted into aspect-ratio buckets.

A bucket is a (width, height) with about resolution x resolution pixels and both sides
divisible by `step`. Every image is resized and center-cropped to the bucket whose shape is
closest to its own, so wide and tall images keep their shape and each batch has one shape.
```

```
"""
from __future__ import annotations
import math
import random
from dataclasses import dataclass
from pathlib import Path

import torch
from PIL import Image, ImageOps
from torch.utils.data import Sampler

IMAGE_EXTENSIONS = {".png", ".jpg", ".jpeg", ".webp", ".bmp"}


@dataclass
class ImageItem:
    path: Path
    caption: str
    bucket: tuple[int, int]          # (width, height) the image is resized to


def make_buckets(resolution: int, step: int = 32, max_ratio: float = 2.0) -> list[tuple[int, int]]:
    """All (width, height) pairs with area <= resolution^2, sides divisible by step and
    width/height between 1/max_ratio and max_ratio."""
    if resolution % step != 0:
        raise ValueError(f"resolution {resolution} must be divisible by {step}")
    area = resolution * resolution
    buckets = {(resolution, resolution)}
    width = resolution
    while True:
        width += step
        height = (area // width) // step * step
        if height < step or width / height > max_ratio:
            break
        buckets.add((width, height))
        buckets.add((height, width))
    return sorted(buckets)


def pick_bucket(width: int, height: int, buckets: list[tuple[int, int]]) -> tuple[int, int]:
    """The bucket whose aspect ratio is closest to the image's (compared on a log scale)."""
    ratio = math.log(width / height)
    return min(buckets, key=lambda b: abs(math.log(b[0] / b[1]) - ratio))


def scan_dataset(dataset_dir, resolution: int, step: int = 32) -> list[ImageItem]:
    """Finds every image that has a caption file next to it (photo.png + photo.txt)."""
    root = Path(dataset_dir)
    if not root.is_dir():
        raise ValueError(f"dataset_dir '{dataset_dir}' is not a folder")
    buckets = make_buckets(resolution, step)
    items, no_caption = [], []
    for path in sorted(root.rglob("*")):
```

```
        if path.suffix.lower() not in IMAGE_EXTENSIONS:
            continue
        caption_file = path.with_suffix(".txt")
        if not caption_file.exists():
            no_caption.append(path.name)
            continue
        caption = caption_file.read_text(encoding="utf-8").strip()
        with Image.open(path) as image:
            width, height = ImageOps.exif_transpose(image).size
        items.append(ImageItem(path, caption, pick_bucket(width, height, buckets)))
    if no_caption:
        raise ValueError(f"{len(no_caption)} images have no .txt caption next to them, "
                         f"for example: {no_caption[:3]}")
    if not items:
        raise ValueError(f"no images found in '{dataset_dir}'")
    return items


def load_image(path, bucket: tuple[int, int]) -> torch.Tensor:
    """Image file -> float32 tensor [3, height, width] in [-1, 1], resized to cover the bucket
    and center-cropped."""
    target_w, target_h = bucket
    with Image.open(path) as image:
        image = ImageOps.exif_transpose(image).convert("RGB")
    scale = max(target_w / image.width, target_h / image.height)
    new_w = max(target_w, round(image.width * scale))
    new_h = max(target_h, round(image.height * scale))
    image = image.resize((new_w, new_h), Image.Resampling.LANCZOS)
    left, top = (new_w - target_w) // 2, (new_h - target_h) // 2
    image = image.crop((left, top, left + target_w, top + target_h))
    pixels = torch.frombuffer(bytearray(image.tobytes()), dtype=torch.uint8)
    pixels = pixels.view(target_h, target_w, 3).permute(2, 0, 1)
    return pixels.float() / 127.5 - 1.0


class GroupedBatchSampler(Sampler):
    """Yields batches of dataset indices in which every item has the same group key (same
    image shape and same caption length), so each batch stacks into one tensor."""

    def __init__(self, keys: list, batch_size: int, shuffle: bool = True, seed: int = 0):
        self.batch_size = batch_size
        self.shuffle = shuffle
        self.random = random.Random(seed)
        self.groups: dict = {}
        for index, key in enumerate(keys):
            self.groups.setdefault(key, []).append(index)

    def __iter__(self):
        batches = []
        for indices in self.groups.values():
            indices = list(indices)
            if self.shuffle:
                self.random.shuffle(indices)
```

```
            batches += [indices[i:i + self.batch_size] for i in range(0, len(indices), self.batch_size)]
        if self.shuffle:
            self.random.shuffle(batches)
        return iter(batches)

    def __len__(self) -> int:
        return sum(math.ceil(len(indices) / self.batch_size) for indices in self.groups.values())
```

Step 7: cache

Latents and text embeddings are computed once and written to disk. The VAE and the text encoder are loaded only when a cache file is missing, and freed before the transformer is loaded. File names contain a fingerprint of the image file, its bucket and the caption text, so a changed image or caption is re-encoded.

NEW FILE: src/amdlora/data/cache.py

```
"""Step 7: precompute image latents and text embeddings to disk, then free the VAE and the
text encoder, so that only the transformer is on the GPU while training.

    latents   one file per image:   <stem>_<fingerprint>.latent.safetensors   {"latents": [128, h, w]}
    text      one file per caption: text_<fingerprint>.safetensors            {"prompt_embeds": [tokens, 20480]}

A fingerprint changes when the image file, its bucket or the caption text changes, so stale
cache files are never reused. Identical captions share one text file.
"""
from __future__ import annotations
import gc
import hashlib
from dataclasses import dataclass
from pathlib import Path

import torch
from safetensors.torch import load_file, save_file
from torch.utils.data import Dataset

from ..models.formats import read_header
from .image_dataset import ImageItem, load_image


@dataclass
class CachedItem:
    latent_file: Path
    text_file: Path
    bucket: tuple[int, int]
    text_tokens: int

    @property
    def group(self) -> tuple:
        """Items with the same group can be stacked into one batch."""
        return (*self.bucket, self.text_tokens)


def _fingerprint(*parts) -> str:
    return hashlib.sha1("|".join(str(p) for p in parts).encode("utf-8")).hexdigest()[:16]


def latent_path(cache_dir, item: ImageItem) -> Path:
    stat = item.path.stat()
    mark = _fingerprint(item.path.resolve(), stat.st_size, stat.st_mtime_ns, *item.bucket)
    return Path(cache_dir) / f"{item.path.stem}_{mark}.latent.safetensors"
```

```
def text_path(cache_dir, prompt: str) -> Path:
    return Path(cache_dir) / f"text_{_fingerprint(prompt)}.safetensors"


def free_gpu() -> None:
    """Gives GPU memory back after the caller has deleted its last reference to a model."""
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()


def build_cache(items: list[ImageItem], extra_prompts: list[str], cfg, family, device) -> list[CachedItem]:
    """Writes every missing cache file and returns one CachedItem per image.
    extra_prompts are cached too (the empty prompt, the sample prompts)."""
    cache_dir = Path(cfg.diffusion.cache_dir or Path(cfg.output_dir) / "cache")
    cache_dir.mkdir(parents=True, exist_ok=True)

    # ---- latents: the VAE is loaded only if something is missing ----
    missing = [item for item in items if not latent_path(cache_dir, item).exists()]
    if missing:
        print(f"cache: encoding {len(missing)} of {len(items)} images")
        vae = family.load_vae(cfg.diffusion.vae, device)
        for item in missing:
            pixels = load_image(item.path, item.bucket)[None]
            latents = family.encode_image(vae, pixels)[0]
            save_file({"latents": latents.float().cpu().contiguous()}, str(latent_path(cache_dir, item)))
        del vae
        free_gpu()

    # ---- text: the text encoder is loaded only if something is missing ----
    prompts = list(dict.fromkeys([item.caption for item in items] + list(extra_prompts)))
    missing_prompts = [p for p in prompts if not text_path(cache_dir, p).exists()]
    if missing_prompts:
        print(f"cache: encoding {len(missing_prompts)} of {len(prompts)} captions")
        text_encoder, tokenizer = family.load_text_encoder(cfg.diffusion.text_encoder, cfg.diffusion.config_dir, device)
        for prompt in missing_prompts:
            embeds = family.encode_prompt(text_encoder, tokenizer, prompt)
            save_file({"prompt_embeds": embeds.cpu().contiguous()}, str(text_path(cache_dir, prompt)))
        del text_encoder
        free_gpu()

    cached = []
    for item in items:
        text_file = text_path(cache_dir, item.caption)
        tokens = read_header(text_file)[0]["prompt_embeds"]["shape"][0]      # read from the header only
        cached.append(CachedItem(latent_path(cache_dir, item), text_file, item.bucket, tokens))
    return cached


def load_prompt_embeds(cache_dir, prompt: str) -> torch.Tensor:
    return load_file(str(text_path(cache_dir, prompt)))["prompt_embeds"]
class CachedDataset(Dataset):
    """Reads the cache files. Each sample is {"latents": [128, h, w], "prompt_embeds": [tokens, 20480]}."""
```

```
def __init__(self, items: list[CachedItem]):
    self.items = items

def __len__(self) -> int:
    return len(self.items)

def __getitem__(self, index: int) -> dict[str, torch.Tensor]:
    item = self.items[index]
    return {
        "latents": load_file(str(item.latent_file))["latents"],
        "prompt_embeds": load_file(str(item.text_file))["prompt_embeds"],
    }
```

Step 8: training loop

A separate loop for image models, so the working LLM loop is not touched. It reuses your optimizer, scheduler, metrics logger and VRAM monitor. Validation uses fixed noise levels and fixed noise, so the number is comparable between runs. Sample images go to `<output_dir>/samples`.

NEW FILE: src/amdlora/train/diffusion.py

```
"""Step 8: LoRA training for image models (flow matching).

One training step:
    1. take cached image latents and cached text embeddings
    2. pick a noise level t and mix the latents with noise
    3. the transformer predicts the velocity (noise - image)
    4. loss = mean squared error between the prediction and the true velocity

Everything model-specific (noise schedule, packing, the transformer call) comes from the family
module, so this loop is the same for every image model.
"""
from __future__ import annotations
import math
import random
import time
from pathlib import Path

import torch
import torch.nn.functional as F
from torch.utils.data import DataLoader

from ..data.cache import CachedDataset, build_cache, load_prompt_embeds
from ..data.image_dataset import GroupedBatchSampler, scan_dataset
from ..io.save import save_lora_comfy
from ..lora.inject import inject_lora, count_parameters
from ..models.loader import load_diffusion
from ..models.registry import get_family
from ..monitor.metrics import MetricsLogger, save_config_copy
from ..monitor.vram import peak_vram_gib, reset_peak_vram
from .optim import build_optimizer, make_scheduler

VAL_TIMES = (0.2, 0.4, 0.6, 0.8)      # fixed noise levels, so validation loss is comparable between runs


def diffusion_loss(transformer, family, batch, device, t=None, noise=None, shift: float = 1.0) -> torch.Tensor:
    latents = batch["latents"].to(device, torch.float32)              # [B, 128, h, w]
    embeds = batch["prompt_embeds"].to(device)                         # [B, tokens, 20480]
    b, _, h, w = latents.shape
    if t is None:
        t = family.sample_timesteps(b, shift, device)
    if noise is None:
        noise = torch.randn_like(latents)
    noisy, target = family.add_noise(latents, noise, t)
    prediction = family.predict(transformer, family.pack(noisy), embeds, t, h, w)
```

```
    return F.mse_loss(prediction.float(), family.pack(target).float())


@torch.no_grad()
def validation_loss(transformer, family, loader, device) -> float:
    """Average loss over the validation images at fixed noise levels and fixed noise."""
    was_training = transformer.training
    transformer.eval()
    total, count = 0.0, 0
    for index, batch in enumerate(loader):
        generator = torch.Generator().manual_seed(1000 + index)
        noise = torch.randn(batch["latents"].shape, generator=generator).to(device)
        for value in VAL_TIMES:
            t = torch.full((batch["latents"].shape[0],), value, device=device)
            total += diffusion_loss(transformer, family, batch, device, t=t, noise=noise).item()
            count += 1
    transformer.train(was_training)
    return total / max(count, 1)


@torch.no_grad()
def write_sample_images(transformer, vae, family, prompts, negative, cfg, cache_dir, out_dir, step: int) -> None:
    d = cfg.diffusion
    was_training = transformer.training
    transformer.eval()
    folder = Path(out_dir) / "samples"
    folder.mkdir(parents=True, exist_ok=True)
    for index, prompt in enumerate(prompts):
        latents = family.generate(transformer, load_prompt_embeds(cache_dir, prompt), negative,
                                  d.resolution, d.resolution, d.sample_steps, d.sample_guidance, d.sample_seed)
        family.to_pil(family.decode_latents(vae, latents)[0]).save(folder / f"step{step:06d}_{index}.png")
    transformer.train(was_training)


def split_items(items: list, val_split: float, seed: int) -> tuple[list, list]:
    """Random train/validation split. With val_split > 0 at least one item is held out."""
    if val_split <= 0 or len(items) < 2:
        return list(items), []
    order = list(range(len(items)))
    random.Random(seed).shuffle(order)
    n_val = min(max(1, round(len(items) * val_split)), len(items) - 1)
    val = sorted(order[:n_val])
    train = sorted(order[n_val:])
    return [items[i] for i in train], [items[i] for i in val]


def make_loader(items, batch_size: int, shuffle: bool, seed: int) -> DataLoader:
    sampler = GroupedBatchSampler([item.group for item in items], batch_size, shuffle, seed)
    return DataLoader(CachedDataset(items), batch_sampler=sampler, num_workers=0)


def train_diffusion(cfg) -> int:
    t, d, mon = cfg.train, cfg.diffusion, cfg.monitor
```

```
family = get_family(d.family)
device = "cuda" if torch.cuda.is_available() else "cpu"
torch.manual_seed(t.seed)
random.seed(t.seed)
out_dir = Path(cfg.output_dir)
out_dir.mkdir(parents=True, exist_ok=True)
cache_dir = Path(d.cache_dir or out_dir / "cache")

# ---- 1. dataset and cache (the VAE and the text encoder are freed again inside) ----
images = scan_dataset(cfg.dataset_dir, d.resolution)
want_samples = bool(mon.sample_every > 0 and mon.sample_prompts)
extra_prompts = [""] + (list(mon.sample_prompts) if want_samples else [])
cached = build_cache(images, extra_prompts, cfg, family, device)
train_items, val_items = split_items(cached, mon.val_split, t.seed)
loader = make_loader(train_items, t.micro_batch_size, shuffle=True, seed=t.seed)
val_loader = make_loader(val_items, t.micro_batch_size, shuffle=False, seed=t.seed) if val_items else None
empty_embeds = load_prompt_embeds(cache_dir, "")
print(f"images: train {len(train_items)} | val {len(val_items)} | batches per epoch: {len(loader)} | "
      f"buckets: {len({item.bucket for item in cached})}")

# ---- 2. transformer and LoRA ----
transformer = load_diffusion(cfg, device)
targets = cfg.lora.target_modules or family.LORA_TARGETS
inject_lora(transformer, targets, cfg.lora.rank, cfg.lora.alpha, cfg.lora.dropout, skip=family.LORA_SKIP)
trainable, _ = count_parameters(transformer)
lora_params = [p for p in transformer.parameters() if p.requires_grad]
print(f"transformer loaded | trainable(LoRA): {trainable / 1e6:.2f}M in {len(lora_params) // 2} layers | device: {device}")
if cfg.memory.gradient_checkpointing:
    transformer.enable_gradient_checkpointing()
transformer.train()
vae = family.load_vae(d.vae, device) if want_samples else None

# ---- 3. optimizer, schedule, run records ----
optimizer = build_optimizer(lora_params, t.optimizer, t.lr)
total_updates = math.ceil(len(loader) / t.grad_accum) * t.epochs
if t.max_steps > 0:
    total_updates = min(total_updates, t.max_steps)
scheduler = make_scheduler(optimizer, t.warmup_steps, total_updates)
save_config_copy(cfg, out_dir)
logger = MetricsLogger(out_dir)
info = {"family": d.family, "resolution": d.resolution}
reset_peak_vram()
best_val_loss, best_step = math.inf, 0

def validate(step: int) -> float:
    nonlocal best_val_loss, best_step
    start = time.perf_counter()
    val_loss = validation_loss(transformer, family, val_loader, device)
    logger.log(kind="val", step=step, val_loss=val_loss)
    note = ""
    if step == 0:
        note = " (baseline, before training)"
    elif val_loss < best_val_loss:
```

```
        best_val_loss, best_step = val_loss, step
        note = " (best so far)"
        if mon.keep_best:
            save_lora_comfy(transformer, out_dir / "lora_best.safetensors", family.to_comfy_name, info)
    print(f"step {step}/{total_updates} | val_loss {val_loss:.4f}{note}")
    return time.perf_counter() - start

def sample(step: int) -> float:
    start = time.perf_counter()
    write_sample_images(transformer, vae, family, mon.sample_prompts, empty_embeds, cfg, cache_dir, out_dir, step)
    print(f"step {step}/{total_updates} | sample images written to {out_dir / 'samples'}")
    return time.perf_counter() - start

# ---- 4. training ----
loss_history: list[float] = []
loss_sum, batches_in_update, updates_done, updates_since_log = 0.0, 0, 0, 0
finished = False
try:
    if val_loader is not None:
        validate(0)
    if want_samples:
        sample(0)
    timer = time.perf_counter()

    for epoch in range(1, t.epochs + 1):
        for batch_index, batch in enumerate(loader):
            if d.caption_dropout > 0 and random.random() < d.caption_dropout:
                batch["prompt_embeds"] = empty_embeds[None].expand(batch["latents"].shape[0], -1, -1)

            loss = diffusion_loss(transformer, family, batch, device, shift=d.timestep_shift)
            (loss / t.grad_accum).backward()
            loss_sum += loss.item()
            batches_in_update += 1
            if batches_in_update % t.grad_accum != 0 and batch_index != len(loader) - 1:
                continue

            torch.nn.utils.clip_grad_norm_(lora_params, t.max_grad_norm)
            lr_used = optimizer.param_groups[0]["lr"]
            optimizer.step()
            scheduler.step()
            optimizer.zero_grad(set_to_none=True)

            loss_history.append(loss_sum / batches_in_update)
            loss_sum, batches_in_update = 0.0, 0
            updates_done += 1
            updates_since_log += 1

            row = {"kind": "train", "step": updates_done, "epoch": epoch, "loss": loss_history[-1], "lr": lr_used}
            if updates_done % t.log_every == 0 or updates_done == total_updates:
                recent = loss_history[-t.log_every:]
                numbers = {"avg_loss": sum(recent) / len(recent),
                           "ms_update": (time.perf_counter() - timer) / max(updates_since_log, 1) * 1000,
                           "vram_gb": peak_vram_gib()}
```

```
                print(f"step {updates_done}/{total_updates} | epoch {epoch}/{t.epochs} | "
                      f"loss {numbers['avg_loss']:.4f} | lr {lr_used:.2e} | "
                      f"{numbers['ms_update']:,.0f} ms/update | peak VRAM {numbers['vram_gb']:.2f} GB")
                row.update(numbers)
                updates_since_log = 0
                timer = time.perf_counter()
            logger.log(**row)

            # validation and sample time is kept out of ms/update
            if val_loader is not None and mon.val_every > 0 and updates_done % mon.val_every == 0:
                timer += validate(updates_done)
            if want_samples and updates_done % mon.sample_every == 0:
                timer += sample(updates_done)
            if t.save_every > 0 and updates_done % t.save_every == 0:
                save_lora_comfy(transformer, out_dir / f"lora_step{updates_done}.safetensors", family.to_comfy_name, info)

            if updates_done >= total_updates:
                finished = True
                break
        if finished:
            break

    if val_loader is not None and (mon.val_every == 0 or updates_done % mon.val_every != 0):
        validate(updates_done)
    if want_samples and updates_done % mon.sample_every != 0:
        sample(updates_done)
finally:
    logger.close()

final_path = out_dir / "lora_final.safetensors"
save_lora_comfy(transformer, final_path, family.to_comfy_name, info)
print(f"saved: {final_path}")
if best_step:
    kept = " -> lora_best.safetensors" if mon.keep_best else ""
    print(f"best val_loss {best_val_loss:.4f} at step {best_step}{kept}")
return 0
```

EDIT: src/amdlora/train/loop.py

Change 1 of 1: find this

```
if cfg.model_family != "llm":
    raise NotImplementedError("Diffusion model training will be done in upcoming steps")
```

and replace it with this

```
if cfg.model_family == "diffusion":
    from .diffusion import train_diffusion
    return train_diffusion(cfg)
if cfg.model_family != "llm":
    raise ValueError(f"Unknown model_family '{cfg.model_family}'")
```

NEW FILE: configs/flux_klein_512.yaml

```
# FLUX Klein LoRA at 512 px, trained from the ComfyUI int8 convrot files.
model_family: diffusion
base_model: E:/Comfy-Desktop/ComfyUI-Shared/models/diffusion_models/flux3-image-klein-base-int8-convrot.safetensors
dataset_dir: data/my_images              # photo.png + photo.txt pairs
output_dir: outputs/flux_klein_run1

diffusion:
  family: flux_klein
  config_dir: E:/Comfy-Desktop/ComfyUI-Shared/models/diffusers/flux3Klein
  vae: E:/Comfy-Desktop/ComfyUI-Shared/models/diffusers/flux3Klein/vae
  text_encoder: E:/Comfy-Desktop/ComfyUI-Shared/models/text_encoders/qwen3vl_4b_int8_convrot.safetensors
  resolution: 512
  timestep_shift: 3.0
  caption_dropout: 0.05
  sample_steps: 20
  sample_guidance: 3.5

lora:
  rank: 16
  alpha: 16
  dropout: 0.0
  target_modules: []                     # empty = the family defaults (attention + feed-forward)

train:
  epochs: 10
  micro_batch_size: 1
  grad_accum: 1
  lr: 1.0e-4
  warmup_steps: 20
  max_grad_norm: 1.0
  optimizer: adamw
  seed: 42
  save_every: 250
  log_every: 10

precision: {compute_dtype: bf16, base_load: auto}
memory: {gradient_checkpointing: true}
amd: {compile: false, tunableop: false}

monitor:
  val_split: 0.1
  val_every: 100
  sample_every: 250
  sample_prompts:
    - "a photo of a red fox sitting in fresh snow"
  keep_best: true
```

Set `dataset_dir` to your image folder. For 1024 px later, change `resolution` to 1024 and use a new `output_dir` (the cache is per output folder).

Step 9: save for ComfyUI, sample script

The LoRA is saved with keys like `diffusion_model.single_blocks.0.q_proj.lora_A.weight`. alpha / rank is multiplied into B, so the file has no alpha keys and every loader applies it at the same strength. Rank and alpha are kept in the file's metadata for reloading.

EDIT: src/amdlora/io/save.py (add at the end of the file)

```
# ---------------------------------------------------------------------------------------
# Image models: ComfyUI / BFL key names
# ---------------------------------------------------------------------------------------
COMFY_PREFIX = "diffusion_model."
QKV_PARTS = ("q_proj", "k_proj", "v_proj")


def _comfy_entries(layer_name: str, layer: LoRALinear, to_comfy_name) -> list[tuple[str, slice]]:
  """The ComfyUI module names one LoRA layer is saved under, each with its rows of B.
  A fused qkv layer is saved as three layers (q_proj, k_proj, v_proj) that share A, which is
  the form ComfyUI and diffusers both read, for fused and unfused base models alike."""
  name = to_comfy_name(layer_name)
  if name.endswith(".qkv"):
    third = layer.B.shape[0] // 3
    return [(f"{name[:-len('.qkv')]}.{part}", slice(i * third, (i + 1) * third)) for i, part in enumerate(QKV_PARTS)]
  return [(name, slice(None))]


def save_lora_comfy(model, path, to_comfy_name, info: dict | None = None) -> int:
  """Saves the LoRA with ComfyUI key names. alpha / rank is multiplied into B, so the file
  needs no alpha keys and every loader applies it at the same strength."""
  layers = find_lora_layers(model)
  if not layers:
    raise ValueError("No Lora Layer was Found in this Model")

  tensors = {}
  for layer_name, layer in layers:
    a = layer.A.detach().cpu().contiguous()
    b = (layer.B.detach() * layer.scale).cpu()
    for name, rows in _comfy_entries(layer_name, layer, to_comfy_name):
      tensors[f"{COMFY_PREFIX}{name}.lora_A.weight"] = a.clone()     # safetensors refuses shared memory
      tensors[f"{COMFY_PREFIX}{name}.lora_B.weight"] = b[rows].contiguous()

  first_layer = layers[0][1]
  metadata = {"rank": str(first_layer.rank), "alpha": str(first_layer.alpha), "format": "amdlora-comfy"}
  metadata.update({key: str(value) for key, value in (info or {}).items()})

  path = Path(path)
  path.parent.mkdir(parents=True, exist_ok=True)
  save_file(tensors, str(path), metadata=metadata)
  return len(layers)


def load_lora_comfy(model, path, to_comfy_name) -> int:
```

```
  """Loads a file written by save_lora_comfy back into the LoRA layers of the model."""
  saved = load_file(str(path))
  layers = find_lora_layers(model)

  loaded = []
  for layer_name, layer in layers:
    entries = _comfy_entries(layer_name, layer, to_comfy_name)
    a_key = f"{COMFY_PREFIX}{entries[0][0]}.lora_A.weight"
    b_keys = [f"{COMFY_PREFIX}{name}.lora_B.weight" for name, _ in entries]
    absent = [key for key in [a_key, *b_keys] if key not in saved]
    if absent:
      raise ValueError(f"Lora File Does Not Match the Model. Missing: {absent[:3]}")
    a = saved[a_key]
    b = torch.cat([saved[key] for key in b_keys], dim=0) / layer.scale
    if a.shape != layer.A.shape or b.shape != layer.B.shape:
      raise ValueError(f"shape mismatch in the layer '{layer_name}' (was it saved with different rank?)")
    loaded.append((layer, a, b))

  with torch.no_grad():
    for layer, a, b in loaded:
      layer.A.copy_(a.to(layer.A.device, layer.A.dtype))
      layer.B.copy_(b.to(layer.B.device, layer.B.dtype))

  return len(layers)
```

NEW FILE: scripts/sample_flux.py

```
"""Generate one image with an image model, with or without a trained LoRA.

    python scripts/sample_flux.py configs/flux_klein_512.yaml --prompt "a red fox in snow" --out fox.png
    python scripts/sample_flux.py configs/flux_klein_512.yaml --prompt "..." --lora outputs/run/lora_final.safetensors
"""
import argparse

from amdlora.config import load_config
from amdlora.env import apply_amd_env


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("config")
    parser.add_argument("--prompt", required=True)
    parser.add_argument("--lora", default="")
    parser.add_argument("--out", default="sample.png")
    parser.add_argument("--steps", type=int, default=0)
    parser.add_argument("--seed", type=int, default=0)
    parser.add_argument("--size", type=int, default=0)
    args = parser.parse_args()

    cfg = load_config(args.config)
    apply_amd_env(cfg.amd)                      # before torch is imported

    import torch
```

```
    from amdlora.data.cache import free_gpu
    from amdlora.io.save import load_lora_comfy
    from amdlora.lora.inject import inject_lora
    from amdlora.models.formats import read_header
    from amdlora.models.loader import load_diffusion
    from amdlora.models.registry import get_family

    d = cfg.diffusion
    family = get_family(d.family)
    device = "cuda" if torch.cuda.is_available() else "cpu"
    size = args.size or d.resolution

    text_encoder, tokenizer = family.load_text_encoder(d.text_encoder, d.config_dir, device)
    prompt_embeds = family.encode_prompt(text_encoder, tokenizer, args.prompt)
    negative_embeds = family.encode_prompt(text_encoder, tokenizer, "")
    del text_encoder
    free_gpu()

    transformer = load_diffusion(cfg, device)
    if args.lora:
        info = read_header(args.lora)[1]           # rank and alpha are stored in the file
        targets = cfg.lora.target_modules or family.LORA_TARGETS
        inject_lora(transformer, targets, int(info["rank"]), float(info["alpha"]), skip=family.LORA_SKIP)
        count = load_lora_comfy(transformer, args.lora, family.to_comfy_name)
        print(f"loaded LoRA: {count} layers, rank {info['rank']}")
    transformer.eval()

    latents = family.generate(transformer, prompt_embeds, negative_embeds, size, size,
                              args.steps or d.sample_steps, d.sample_guidance, args.seed)
    vae = family.load_vae(d.vae, device)
    family.to_pil(family.decode_latents(vae, latents)[0]).save(args.out)
    print(f"saved: {args.out}")


if __name__ == "__main__":
    main()
```

NEW FILE: tests/test_save_comfy.py

```
import pytest
import torch
import torch.nn as nn
from safetensors.torch import load_file
from amdlora.io.save import load_lora_comfy, save_lora_comfy
from amdlora.lora.inject import inject_lora
from amdlora.models.flux_klein import to_comfy_name


class Attention(nn.Module):
    def __init__(self):
        super().__init__()
        self.to_qkv = nn.Linear(8, 24, bias=False)
        self.to_out = nn.ModuleList([nn.Linear(8, 8, bias=False)])
```

```
class Tiny(nn.Module):
    def __init__(self):
        super().__init__()
        self.single_transformer_blocks = nn.ModuleList([nn.ModuleDict({"attn": Attention()})])


def make_model(rank: int = 4, alpha: float = 8.0):
    model = Tiny()
    inject_lora(model, ["attn.to_qkv", "attn.to_out.0"], rank=rank, alpha=alpha)
    return model


def randomize_b(model) -> None:
    with torch.no_grad():
        for module in model.modules():
            if hasattr(module, "B"):
                module.B.normal_()


def test_keys_follow_the_comfyui_names(tmp_path):
    model = make_model()
    path = tmp_path / "lora.safetensors"
    assert save_lora_comfy(model, path, to_comfy_name, {"family": "flux_klein"}) == 2
    saved = load_file(str(path))
    block = "diffusion_model.single_blocks.0"
    expected = {f"{block}.{name}.lora_{side}.weight"
                for name in ("q_proj", "k_proj", "v_proj", "attn_out") for side in ("A", "B")}
    assert set(saved) == expected
    assert saved[f"{block}.q_proj.lora_B.weight"].shape == (8, 4)        # one third of the fused rows
    assert saved[f"{block}.q_proj.lora_A.weight"].shape == (4, 8)


def test_saved_file_gives_the_same_weight_change(tmp_path):
    torch.manual_seed(0)
    model = make_model()
    randomize_b(model)
    qkv = model.single_transformer_blocks[0]["attn"].to_qkv
    path = tmp_path / "lora.safetensors"
    save_lora_comfy(model, path, to_comfy_name)
    saved = load_file(str(path))
    block = "diffusion_model.single_blocks.0"
    rebuilt = torch.cat([saved[f"{block}.{part}.lora_B.weight"] @ saved[f"{block}.{part}.lora_A.weight"]
                         for part in ("q_proj", "k_proj", "v_proj")], dim=0)
    assert torch.allclose(rebuilt, qkv.delta_weight(), atol=1e-6)        # alpha / rank is already inside B


def test_round_trip(tmp_path):
    torch.manual_seed(0)
    model = make_model()
    randomize_b(model)
    path = tmp_path / "lora.safetensors"
    save_lora_comfy(model, path, to_comfy_name)
    fresh = make_model()
```

```
    assert load_lora_comfy(fresh, path, to_comfy_name) == 2
    for a, b in zip(model.parameters(), fresh.parameters()):
        if a.requires_grad:
            assert torch.allclose(a, b, atol=1e-6)

    with pytest.raises(ValueError):
        load_lora_comfy(make_model(rank=2), path, to_comfy_name)
```

NEW FILE: tests/test_image_dataset.py

```
import pytest
import torch
from PIL import Image
from amdlora.data.cache import CachedItem, latent_path, text_path
from amdlora.data.image_dataset import (GroupedBatchSampler, ImageItem, load_image, make_buckets,
                                        pick_bucket, scan_dataset)
from amdlora.train.diffusion import split_items


def test_buckets():
    buckets = make_buckets(512)
    assert (512, 512) in buckets
    for width, height in buckets:
        assert width % 32 == 0 and height % 32 == 0
        assert width * height <= 512 * 512 and max(width, height) / min(width, height) <= 2.0
    assert pick_bucket(1000, 1000, buckets) == (512, 512)
    wide = pick_bucket(1600, 900, buckets)
    assert wide[0] > wide[1]
    assert pick_bucket(900, 1600, buckets) == (wide[1], wide[0])


def test_scan_and_load(tmp_path):
    Image.new("RGB", (300, 200), (255, 0, 0)).save(tmp_path / "a.png")
    (tmp_path / "a.txt").write_text(" a red image \n", encoding="utf-8")
    items = scan_dataset(tmp_path, 256)
    assert len(items) == 1 and items[0].caption == "a red image"
    width, height = items[0].bucket
    assert width > height

    pixels = load_image(items[0].path, items[0].bucket)
    assert pixels.shape == (3, height, width) and pixels.dtype == torch.float32
    assert torch.allclose(pixels[0], torch.ones(height, width))
    assert torch.allclose(pixels[1], -torch.ones(height, width))


def test_missing_caption_raises(tmp_path):
    Image.new("RGB", (64, 64)).save(tmp_path / "a.png")
    with pytest.raises(ValueError, match="caption"):
        scan_dataset(tmp_path, 256)


def test_grouped_batches_never_mix_groups():
    keys = ["a"] * 5 + ["b"] * 3
```

```
    sampler = GroupedBatchSampler(keys, batch_size=2, shuffle=True, seed=1)
    batches = list(sampler)
    assert len(batches) == len(sampler) == 5
    assert sorted(i for batch in batches for i in batch) == list(range(8))
    for batch in batches:
        assert len({keys[i] for i in batch}) == 1


def test_cache_names_change_with_the_content(tmp_path):
    Image.new("RGB", (64, 64)).save(tmp_path / "a.png")
    item = ImageItem(tmp_path / "a.png", "caption", (256, 256))
    other = ImageItem(tmp_path / "a.png", "caption", (288, 224))
    assert latent_path(tmp_path, item) != latent_path(tmp_path, other)
    assert text_path(tmp_path, "one") != text_path(tmp_path, "two")
    assert text_path(tmp_path, "one") == text_path(tmp_path, "one")
    assert CachedItem(tmp_path / "x", tmp_path / "y", (256, 256), 80).group == (256, 256, 80)


def test_split_items():
    items = list(range(10))
    train, val = split_items(items, 0.2, seed=1)
    assert len(val) == 2 and sorted(train + val) == items
    assert split_items(items, 0.0, seed=1) == (items, [])
    assert len(split_items(items, 0.01, seed=1)[1]) == 1            # at least one when asked for
```

Final checks

```
# 1. all unit tests (expected: 102 passed)
pytest -q

# 2. base model generates an image through the new loader
python scripts/sample_flux.py configs/flux_klein_512.yaml --prompt "a photo of a red fox sitting in fresh snow" --out base.png

# 3. train (set dataset_dir in the yaml first)
amdlora train configs/flux_klein_512.yaml

# 4. same prompt with the LoRA
python scripts/sample_flux.py configs/flux_klein_512.yaml --prompt "..." --lora outputs/flux_klein_run1/lora_final.safetensors --out lora.png
```

Then copy `lora_final.safetensors` into ComfyUI's `models/loras` folder and load it with a Load LoRA node on the Klein model. That is the one check I could not run for you.

Numbers to expect on your RX 7800 XT (512 px, batch 1, rank 16)

Phase	Time	Peak VRAM
Text encoder load + encode	12 s + under 1 s per caption	3.8 GB
Transformer load	22 s	5.5 GB

Phase	Time	Peak VRAM
Training update (gradient checkpointing on)	1.7 s	7.3 to 7.5 GB
Sample image, 20 steps with guidance	23 s	6.3 GB